

Tazama Demo UI - Update Plan

1. Current State Analysis

1.1 Application Overview

The Tazama Demo UI is a **Next.js 14** web application that provides a visual interface for demonstrating the Tazama real-time fraud detection pipeline. It shows debtors and creditors initiating ISO 20022 payment transactions, and visualises the live Tazama processing results across rules, typologies, and the **event-adjudicator**.

Next.js is a React-based web framework developed by Vercel. It extends plain React (a JavaScript library for building user interfaces) by adding features that React alone does not provide: server-side rendering (generating page HTML on the server before sending it to the browser), a file-system-based routing system (the folder structure under **app/** defines the application's URLs), and built-in API route handling (serverless-style functions that run on the server, defined as **route.ts** files alongside the pages they serve). The "**App Router**" refers to the newer of Next.js's two routing systems (introduced in Next.js 13), which is based on React Server Components and replaces the older "Pages Router".

Current version: **3.0.0** (per **package.json**)

1.2 Technology Stack

Layer	Technology	Purpose / Notes
Framework	Next.js 14 (App Router)	Full-stack React framework. Handles page routing, server-side rendering, and API endpoints within one project. Currently locked to 14.2.31 in yarn.lock . Upgrade to 15.5.18 required - see Section 6 for details.
Language	TypeScript	A superset of JavaScript that adds static type checking. Catches type mismatches at compile time rather than at runtime.
Styling	Tailwind CSS + SCSS modules	Tailwind CSS is a utility-first CSS framework - instead of writing custom CSS rules you compose styles using short class names directly in HTML/JSEX (e.g. className="flex rounded-lg bg-gray-100"). SCSS modules are scoped CSS files used for the few components that require more traditional stylesheet authoring.
State management	React Context API with custom reducers	React Context is a built-in React mechanism for sharing state across components without passing it through every level of the component tree ("prop drilling"). The app wraps the component tree in EntityProvider and ProcessorProvider context providers that hold and update all application state. A reducer is a pure function that describes how state changes in response to an event. It takes two inputs - the current state and a description of what just happened (called an action , e.g. SET_RULES_SUCCESS) - and returns the new

Layer	Technology	Purpose / Notes
		state. This keeps all state change logic in one predictable place rather than scattered across components.
Real-time	Socket.IO (server + client)	A library that provides persistent, bidirectional communication between the browser and the server over WebSockets (falling back to HTTP long-polling if WebSockets are unavailable). The server pushes Tazama processing results to the browser as they arrive, rather than the browser polling repeatedly.
Message broker	NATS (via <code>nats</code> npm package)	NATS is a lightweight publish/subscribe messaging system. Tazama services publish their processing results (rule outcomes, typology scores, <code>event-adjudicator</code> decisions) to NATS topics. The demo UI server subscribes to those topics and forwards the messages to the browser via Socket.IO.
Database	PostgreSQL (via <code>pg</code> pool)	The demo UI currently reads the Tazama network map, rule result bands, and typology thresholds directly from PostgreSQL using the <code>pg</code> npm package. This direct database dependency will be eliminated in the updated architecture - see Section 3 and Section 5 - replaced by API calls to the admin-service, which already owns and exposes this data.
Message codec	<code>@tazama-lf/frms-coe-lib</code> protobuf helpers	Protocol Buffers (protobuf) is a binary serialisation format developed by Google - more compact and faster to parse than JSON. Tazama services communicate over NATS using protobuf-encoded messages. The <code>frms-coe-lib</code> library (Tazama's shared "core" library) provides a <code>decode()</code> helper that converts the raw binary NATS message bytes back into a readable JavaScript object.
HTTP client	Axios	A promise-based HTTP client used to make REST API calls from the browser (e.g., to the Tazama TMS and Admin Service). Similar to the browser's built-in <code>fetch</code> but with additional features like automatic JSON parsing and request interceptors.
Custom server	Node.js / Express (<code>server.js</code>)	Next.js normally runs on its own built-in HTTP server. This application replaces that with a custom server script (<code>server.js</code>) built on Node.js (the JavaScript runtime) so that Socket.IO and NATS subscription logic can be co-located in the same process. Express is a minimal Node.js web framework, though in this case it is not used directly - the <code>server.js</code> uses the lower-level Node.js <code>http.createServer()</code> instead.

1.3 Architecture - What Exists Today

A few terms used throughout this section:

- **"use client" / client component** - A React component marked with `"use client"` at the top of its file runs its JavaScript code in the browser. It has access to browser-only APIs such as `localStorage`,

`window`, and event listeners.

- **Server component** - A React component that runs only on the server (the default in Next.js App Router). It can read files, query databases, and access secrets without exposing them to the browser, but it cannot use browser-only APIs.
- **API Route** - A `route.ts` file inside `app/api/` that behaves like a small HTTP endpoint (e.g., `GET /api/rules`). It runs server-side and can safely access databases and credentials.
- **localStorage** - A browser-side key/value store that persists data between page refreshes. It is visible to all JavaScript running on the same domain, including any injected third-party scripts, and is never appropriate for storing credentials.
- **SSR (Server-Side Rendering)** - The process of generating a page's HTML on the server and sending it to the browser ready-rendered, rather than sending an empty shell and rendering entirely in the browser.

```

Browser
|
+--> Next.js Pages (React, "use client") -- localStorage for config
|
|   +--> Socket.IO client (real-time results)
|   +--> Axios calls to Next.js API routes
|
Next.js API Routes (server-side, thin)
|
|   +--> pg Pool (direct PostgreSQL)
|
Custom server.js (Node.js / Socket.IO server)
|
|   +--> NATS subscriptions (raw connection, URL from browser via socket
message)
|   +--> Serves Next.js via next.prepare()

```

Key structural problems identified:

1. **No front-end / back-end separation.** The custom `server.js` fuses the Next.js SSR server and the NATS subscriber into one process. There is no dedicated API backend. All "backend" logic lives inside a single Node.js process that also renders React pages.

Why this matters - reliability: Because the NATS subscriber and the web server share a process, a crash or unhandled exception in the NATS layer takes down the UI for all users, and vice versa. There is no fault isolation between the two concerns.

Why this matters - security: This is not merely a design preference. The fused architecture is the *root cause* of problems 2, 3, and 4 below, not a separate issue alongside them. Because `server.js` has no independent server-side configuration, it has no way to know what NATS endpoint to connect to without being told. The only available channel for that information is the browser - hence `socket.on("uiconfig")`, hence the NATS URL in `localStorage`, hence the credentials in `NEXT_PUBLIC_*` variables. The security vulnerabilities are structural consequences of this decision, not incidental mistakes. Fixing them without fixing the architecture means patching symptoms while the cause remains.

Is this acceptable because it is a demo? For a demo that runs locally in a controlled environment and is never exposed to external users, the shortcuts are arguably tolerable. That is no longer the case here. The Beta Sandbox is a test environment - no real or private data is involved - but it is publicly reachable by external tenants over the internet. "Demo" describes the *purpose* of the UI (showing the platform to prospective users), not the threat model it operates under. A publicly accessible application, even one handling only synthetic test data, must be defensible against malicious external users. An attacker does not stop probing because the data is fake; the NATS injection vector and credential exposure are exploitable regardless of what is in the database.

2. **Sensitive credentials exposed to the browser.** Database credentials (`PG_HOST`, `PG_PORT`, `PG_USER`, `PG_PASSWORD`, `PG_DATABASE`) are prefixed `NEXT_PUBLIC_*`, which means they are embedded in the browser JavaScript bundle. This is a critical OWASP A02:2021 (Cryptographic Failures / Sensitive Data Exposure) issue.

Why this matters: Anyone who opens browser dev tools (F12 > Sources) can read these values in plain text - they are baked into the JavaScript bundle the browser downloads before the page renders. These are not scoped API tokens; they are direct database credentials that grant read/write access to PostgreSQL, bypassing every application-level access control. An attacker with these values can connect to the database directly from anywhere.

Is this acceptable? No. There is no context - demo, internal, or otherwise - in which database credentials belong in a browser-downloadable JavaScript file. The `NEXT_PUBLIC_` prefix is a deliberate Next.js mechanism for making values available to browser code; using it for credentials is the exact opposite of its intended use.

3. **UI configuration stored in `localStorage`.** The settings page stores TMS API URL, NATS URL, admin service URL, NATS credentials, and database credentials in plain-text `localStorage`. This is accessible to any script on the page and is inappropriate for a production or sandbox deployment.

Why this matters: `localStorage` is readable by any JavaScript executing in the same browser origin. A cross-site scripting (XSS) vulnerability - even a minor one in a single third-party dependency - can silently exfiltrate every key/value stored there. Unlike `HttpOnly` cookies, `localStorage` has no mechanism to block JavaScript access. Stored values are also permanent: they survive page refreshes and browser restarts indefinitely and are never automatically expired or rotated.

Is this acceptable? For non-sensitive user preferences (theme, language), `localStorage` is fine. For credentials - NATS passwords, database passwords, API keys - it is not acceptable under any circumstances. The issue is not the demo context; `localStorage` is structurally the wrong storage mechanism for secrets regardless of deployment.

4. **NATS URL is controlled by the browser.** The custom server connects to NATS using a URL that the browser sends via a Socket.IO message (`socket.on("uiconfig", ...)`). A malicious client could redirect the server's NATS connection to an arbitrary host.

Why this matters: This is a Server-Side Request Forgery (SSRF) vulnerability. The server process, running inside the deployment network, will attempt a TCP connection to any host the browser names. This can be used to probe internal hosts that are not publicly reachable, to reach cloud metadata services (e.g. `http://169.254.169.254` on AWS, which can yield IAM credentials), or to redirect the

NATS connection to an attacker-controlled host that reads or injects messages into the pipeline. The connection is attempted before any authentication check, with no URL validation.

Is this acceptable? No. This is not a theoretical risk. The code as written executes an unconditional TCP connection to whatever host the browser sends. Combined with the absence of authentication (problem 5), any internet user can trigger this.

5. **No authentication or authorisation.** Any user who can reach the URL can see and interact with the demo UI and, by extension, submit transactions to the connected Tazama instance.

Why this matters: Without authentication, the demo UI is an open, anonymous interface to the Tazama pipeline. Any internet user who discovers the URL can submit arbitrary transactions, observe all processing results, and - via problem 4 - trigger SSRF from the server. Even in a test environment with no real data, anonymous write access creates noise that obscures legitimate test results, can generate significant processing load, and gives an attacker a detailed picture of pipeline behaviour.

Is this acceptable? The need to support both a local developer demo (no auth) and a public sandbox (auth required) is the same pattern addressed by other forward-facing Tazama components, which make authentication optional via a configuration flag (e.g. `AUTHENTICATED=true/false`). The correct solution here is the same: implement authentication and gate it behind a flag, so the application can run unauthenticated locally and with full auth enforced in the sandbox - without maintaining two codebases. Multi-tenancy (problem 6) also depends on authentication: without identity there is no basis for tenant routing.

6. **No multi-tenancy.** The application is designed for a single Tazama deployment. There is no concept of users, organisations, or tenants.

Why this matters: Without tenancy isolation, all connected users share a single Tazama instance and a single set of NATS topics. Every transaction any user submits generates results that are broadcast to every other connected browser. In a sandbox with multiple tenants, each tenant's test activity is visible to all others.

Is this acceptable? For a single-tenant internal demo this is an expected constraint, not a flaw. It is a hard blocker for the Beta Sandbox multi-tenant deployment. It is not a standalone security issue, but it becomes one in combination with problem 5 - without identity there is no mechanism to enforce per-tenant data separation. The good news is that multi-tenancy is largely transparent to the demo UI: admin-service and TMS handle tenant data isolation themselves by extracting the `tenantId` from the JWT on every inbound request. The demo UI's job is simply to authenticate the user, store their JWT, and forward it on API calls. The no-auth/single-tenant local case works identically - downstream services inject `tenantId = "DEFAULT"` in their own middleware when `AUTHENTICATED=false`. The one area where the demo UI itself may need tenant-awareness is the real-time NATS layer, which depends on how the platform publishes messages (see Q2 / Q11).

7. **Middleware is boilerplate-only.** `middleware.ts` contains a redirect for a Blazity boilerplate demo domain and a comment `// TODO: Feel free to remove this block`. It provides no real functionality.

Why this matters: A `middleware.ts` file that exists but does nothing creates a false impression of route protection. A developer adding authentication later may assume the middleware infrastructure is already in place and correctly wired, when in fact the placeholder has no meaningful behaviour. This is

the same class of problem as the Next.js CVE-2025-29927 bypass: middleware that appears to protect routes but does not.

Is this acceptable? The boilerplate itself does no immediate harm. Leaving a vestigial TODO placeholder in a repo heading toward a public deployment is poor hygiene - it should either be replaced with real route protection or deleted. It is a low-severity hygiene issue in its current state, but becomes a security issue the moment someone assumes it is doing something it is not.

8. **Tazama 3.0.0 assumptions.** The message structures (`PACS008`, `PACS002`, `event-adjudicator` result shapes) and the Admin Service interaction are coded against the Tazama 3.0.0 API contract. Tazama 4.0.0 introduces changes to these contracts.

Why this matters: The most likely failure mode is silent. Fields that were renamed in 4.0.0 will be `undefined` at runtime, causing the UI to render as if processing produced no results - no rule lights, no typology scores, no alert/interdiction state - without throwing any visible error. Silent failures are worse than visible ones: they are harder to diagnose and may not be noticed during a live demonstration.

Is this acceptable? For demonstrating against a 3.0.0 deployment, the current code is correct. For the Beta Sandbox running Tazama 4.0.0 it is a functional blocker. This is a correctness issue, not a security one, but it must be resolved before go-live.

9. **No integration with Tazama core libraries beyond protobuf.** The `frms-coe-lib` is only used for protobuf decoding in `server.js`. None of the shared type definitions, validators, or helper utilities from the core library are used in the UI code.

Why this matters: The inline type interfaces in `entity.interface.tsx` are manually maintained copies of structures that `frms-coe-lib` defines authoritatively. When Tazama data contracts change, the library exports updated types - but the demo UI's copies will not update automatically. TypeScript will not catch the drift because it validates the application code against the inline copy, not the canonical source. The result is type-safe code that is semantically wrong.

Is this acceptable? For a one-off demo that is never updated, this is a tolerable shortcut. For a codebase that needs to track Tazama major version upgrades it is an ongoing maintenance liability: every release requires a manual audit of all inline type copies instead of simply bumping a dependency.

10. **Storybook / Playwright tests are broken or vestigial.** The CI workflow runs Storybook acceptance tests, but no Storybook stories exist in the current codebase. The `check.yml` workflow references `yarn build-storybook` which will fail.

Why this matters: `yarn build-storybook` fails in CI because there are no stories to build. This means every PR either produces a failed CI run, or the step is being silently ignored. A CI pipeline that consistently fails - or is routinely overridden - provides no quality signal and trains contributors to dismiss CI failures as expected noise.

Is this acceptable? Broken CI is not acceptable in any context, including a demo repo. If Storybook is not being used, the step must be removed. The current state - a workflow step that references infrastructure that does not exist - indicates that the CI configuration has not been maintained alongside the code.

2. Goals for the Updated Application

Goal	Description
G1	Support multi-tenant authentication for any Tazama stack deployment
G2	Update message contracts and API calls to Tazama 4.0.0
G3	Establish a clean front-end / back-end architecture
G4	Integrate Tazama core libraries (<code>frms-coe-lib</code>) for types and utilities
G5	Eliminate all security vulnerabilities identified above
G6	Maintain the existing demo UX (visual pipeline with rule/typology lights)
G7	Make the application deployable as a component of any Tazama full-stack deployment

3. Proposed Architecture

3.1 Layered Architecture

The proposed architecture follows a **Backend for Frontend (BFF)** pattern. A BFF is a server-side layer that sits between the browser and the downstream services (Tazama APIs, database, NATS). It has two key properties: it is purpose-built for its specific frontend (so it speaks the exact shape of data the UI needs), and it is the only layer that holds credentials. The browser talks only to the BFF; the BFF talks to everything else on the browser's behalf.

```
Browser (React / Next.js client components)
|
+--> Next.js Server Components (SSR, auth session reads)
+--> /api/* Next.js Route Handlers (BFF layer)
    |
    +--> Tazama TMS API (send transactions)
    +--> Tazama Admin Service API (conditions, network map, rules,
typologies)
    +--> NATS / WebSocket proxy (real-time results)
|
Auth layer (NextAuth.js with Keycloak provider)
```

The demo UI holds **no database credentials** - all configuration data (network map, rule bands, typology thresholds, conditions) is retrieved from the admin-service API by forwarding the authenticated user's JWT. admin-service scopes all responses to the user's `tenantId` automatically (see Section 3.2).

The key principle is that the browser **never** connects directly to Tazama APIs, NATS, or the database. All external communication is proxied through Next.js API Route Handlers that run server-side. Credentials live only in server-side environment variables (no `NEXT_PUBLIC_` prefix for anything sensitive).

3.2 Authentication - Multi-Tenant Design

Some terms used in this section:

- **Keycloak** - An open-source Identity and Access Management (IAM) server. It handles user login, password storage, and token issuance so the application does not have to. Tazama already uses Keycloak for its internal service authentication.
- **OIDC (OpenID Connect)** - A standard login protocol built on top of OAuth 2.0. The browser redirects the user to the Keycloak login page; after a successful login Keycloak redirects back with a short-lived **authorisation code** that the server exchanges for tokens.
- **JWT (JSON Web Token)** - A compact, signed token that Keycloak issues after login. It contains structured **claims** - key/value fields describing the authenticated user (name, email, roles, custom attributes like `tenant_id`). Because JWTs are cryptographically signed by Keycloak, the application can verify their authenticity without calling Keycloak again on every request.
- **NextAuth.js (Auth.js v5)** - A Next.js authentication library that handles the OIDC login/logout flow, stores the session securely in a server-side encrypted cookie, and exposes the authenticated user's data to server components and API routes via `getServerSession()`.
- **Realm** - In Keycloak, a realm is an isolated namespace for users, roles, and clients. Tenants can be separated by giving each its own realm, or they can share a realm with their users distinguished by roles or groups.
- **Multi-tenancy** - The ability for a single deployed instance of the application to serve multiple independent organisations ("tenants"), each seeing only their own data and connecting to their own Tazama backend.
- **@tazama-lf/auth-lib** - The shared Tazama authentication library. Provides `extractTenant(AUTHENTICATED, authHeader)`, which is the standard entry-point for all Tazama forward-facing components. When `AUTHENTICATED=false` it returns `{ success: true, tenantId: 'DEFAULT' }` with no header required. When `AUTHENTICATED=true` it validates the Bearer token and extracts the `tenantId` claim; returns `{ success: false }` on any failure.

Reference implementation: `tazama-lf/batch-ppa` [PR #227](#) (`feat: add authenticated execute support with tenant propagation`) establishes the canonical pattern for `AUTHENTICATED` flag + `extractTenant` + `DEFAULT` tenant fallback. The demo UI authentication and multi-tenancy implementation should follow this pattern directly rather than reinventing it.

Authentication in any Tazama stack deployment uses **Keycloak** as the identity provider. Keycloak is a standard component of the Tazama full stack (as defined in `tazama-full-stack-docker`); the demo UI connects to whichever Keycloak server is configured in its `KEYCLOAK_ISSUER` env var (e.g. `http(s)://<keycloak-host>/realms/Tazama`). The Next.js application integrates via **NextAuth.js** (Auth.js v5) for the browser-facing OIDC login flow. The BFF API Route Handlers use `@tazama-lf/auth-lib`'s `extractTenant` to validate and propagate the `tenantId` on every request, consistent with other Tazama components.

Multi-tenancy is modelled as follows:

- All tenants share a single **Tazama realm** on the deployment's Keycloak server. Users belong to a group hierarchy - functional group → role sub-group → tenant sub-group → individual user - from which Keycloak derives their assigned roles and `tenantId` JWT claim. The demo UI requires no knowledge of this hierarchy; it reads `tenantId` from the JWT via `@tazama-lf/auth-lib`'s `extractTenant`. Demo UI users must belong to a functional group (e.g. `Tazama-Demo`) whose role sub-groups grant the privileges required by both the TMS API and the admin-service read endpoints - the BFF forwards the user's JWT to both services, so the user's token must satisfy the privilege check on each. Unlike TCS/TRS (server-to-server components with no user sessions), the demo UI has authenticated user sessions, making the user JWT the natural and sufficient credential for all downstream API calls.

- At login, the user's JWT contains a `tenantId` claim (defined in `@tazama-lf/auth-lib`'s `TazamaToken` interface). NextAuth stores this in the server-side session cookie.
- The BFF API Route Handlers do not unpack or route by `tenantId`. Their job is to verify the user has a valid session, extract the JWT from the session, and forward it as the `Authorization: Bearer <jwt>` header on all calls to admin-service and TMS.
- admin-service and TMS each run their own `validateTenantMiddleware`, which calls `extractTenant(AUTHENTICATED, req.headers.authorization)` internally. They resolve `tenantId` from the JWT and use it to scope all their database queries. This is transparent to the demo UI.
- In local dev mode (`AUTHENTICATED=false`), the demo UI sends no Authorization header; admin-service and TMS inject `tenantId = 'DEFAULT'` in their own middleware automatically.
- The TMS and admin-service API endpoints are the same for all tenants - there are no per-tenant URLs. Tenant data isolation is handled server-side by the APIs themselves.

Authentication flow (`AUTHENTICATED=true` - deployed stack):

1. User visits demo UI -> middleware redirects to /login if no session (sign-in page is new - no current login page exists in the codebase)
2. User authenticates via Keycloak (OIDC via NextAuth)
3. NextAuth stores JWT in an encrypted server-side session cookie
4. Browser makes API call (session cookie sent automatically)
BFF route handler:
 - validates session exists via `auth()` (`Auth.js v5`)
 - extracts JWT from `session.accessToken`
 - forwards request to admin-service / TMS with `Authorization: Bearer <jwt>`
5. admin-service / TMS run their own `validateTenantMiddleware`:
`extractTenant(AUTHENTICATED, req.headers.authorization)`
-> resolves `tenantId` from JWT, scopes all DB queries
6. On logout, client calls NextAuth `signOut()` which clears the session cookie.
Server-side Keycloak session termination is out of scope.

Authentication flow (`AUTHENTICATED=false` - local developer mode):

1. User visits demo UI -> no auth check, page loads directly
2. BFF route handlers forward requests to admin-service / TMS with no Authorization header.
admin-service / TMS are also running with `AUTHENTICATED=false`;
their `validateTenantMiddleware` injects `tenantId = 'DEFAULT'` automatically.
3. APIs respond with DEFAULT tenant data.

Tenant isolation:

- Tenant data isolation is handled entirely by admin-service and TMS. Each API call scoped by the `tenantId` the service extracts from the JWT. The demo UI has no visibility into this scoping.
- The browser receives only sanitised data (results, network map); it never receives URLs, API keys, or NATS credentials.

3.3 Real-time Architecture

The current approach of having the server accept a NATS URL from the browser via Socket.IO is a security vulnerability and must be replaced.

NATS subject naming - verified from source

NATS subject names in the Tazama pipeline are deterministically derived from the network map's `rule.id` and `typology.cfg` values. This is implemented in `frms-coe-lib/src/helpers/networkMapIdentifiers.ts` and verified against event-director, rule-executer, typology-processor, and transaction-aggregation-decisioning-processor:

Stage	Subject formula	Example
event-director → rule-executer	<code>sub-rule-\${rule.id}</code>	<code>sub-rule-DEFAULT-901@1.0.0</code>
rule-executer → typology-processor	<code>pub-rule-\${rule.id}</code>	<code>pub-rule-DEFAULT-901@1.0.0</code>
typology-processor → event-adjudicator	<code>typology-\${typology.cfg}</code>	<code>typology-001@1.0.0</code>
event-adjudicator → alert output	<code>\${ALERT_PRODUCER}</code> or <code>\${ALERT_PRODUCER}-\${tenantId}</code>	<code>investigation-service</code> or <code>investigation-service-org-xyz</code>
typology-processor → interdiction output	<code>\${TP_INTERDICTION_PRODUCER}</code> or <code>\${TP_INTERDICTION_PRODUCER}-\${tenantId}</code>	<code>interdiction-service-tp</code> or <code>interdiction-service-tp-org-xyz</code>
event-flow processor → interdiction output	<code>\${EF_INTERDICTION_PRODUCER}</code> or <code>\${EF_INTERDICTION_PRODUCER}-\${tenantId}</code>	<code>interdiction-service-ef</code> or <code>interdiction-service-ef-org-xyz</code>

where `rule.id` = `${RULE_NAME}@${RULE_VERSION}` and `typology.cfg` is the typology's `cfg` field from the network map. The three terminal subjects are each controlled by a pair of environment variables in the producing service: `ALERT_PRODUCER` / `ALERT_DESTINATION` in event-adjudicator, and `INTERDICTION_PRODUCER` / `INTERDICTION_DESTINATION` in both typology-processor and event-flow processor. When `*_DESTINATION=tenant` the `-${tenantId}` suffix is appended; when `*_DESTINATION=global` (the default) the bare producer string is used. The `tenantId` used in the suffix comes from `BaseMessage.TenantId` in the transaction payload, not from the JWT - but they refer to the same value. Default values from `docker-compose.hub.relay.yaml`: `ALERT_PRODUCER=investigation-service`, typology-processor `INTERDICTION_PRODUCER=interdiction-service-tp`, event-flow processor `INTERDICTION_PRODUCER=interdiction-service-ef`; all three default to `*_DESTINATION=global`.

Because the demo UI subscribes to all three terminal subjects from a single process, it uses distinct env var names to avoid ambiguity: `ALERT_PRODUCER` / `ALERT_DESTINATION` (event-adjudicator),

TP_INTERDICTION_PRODUCER / TP_INTERDICTION_DESTINATION (typology-processor), and EF_INTERDICTION_PRODUCER / EF_INTERDICTION_DESTINATION (event-flow processor). The subject computation in the demo UI mirrors the exact pattern from `typology-processor/src/logic.service.ts`:

```
const alertSubject = config.ALERT_DESTINATION === 'tenant'
  ? `${config.ALERT_PRODUCER}-${tenantId}` : config.ALERT_PRODUCER
const tpInterdictionSubject = config.TP_INTERDICTION_DESTINATION === 'tenant'
  ? `${config.TP_INTERDICTION_PRODUCER}-${tenantId}` :
  config.TP_INTERDICTION_PRODUCER
const efInterdictionSubject = config.EF_INTERDICTION_DESTINATION === 'tenant'
  ? `${config.EF_INTERDICTION_PRODUCER}-${tenantId}` :
  config.EF_INTERDICTION_PRODUCER
```

The current demo UI hard-codes subscriptions to `investigation-service` and `interdiction-service-tp`, and has no subscription to `interdiction-service-ef` at all. The subject naming formula was always correct, but the demo UI bypassed it by hard-coding two of the three downstream names rather than deriving them from configuration.

NATS authentication - current state

NATS authentication in the Tazama platform is distinct from the Socket.IO session authentication described above, and is worth addressing explicitly because the demo UI's situation is different from the other Tazama services.

Every other Tazama service (event-director, rule-executer, typology-processor, event-adjudicator, etc.) connects to NATS using `{ servers: startupConfig.serverUrl }` - the `IStartupConfig` interface has no separate username, password, NKey, or credentials fields. This means any NATS credentials must be embedded in the `SERVER_URL` environment variable itself (e.g. `nats://user:pass@host:4222`). The `.env.template` files show `SERVER_URL=0.0.0.0:4222` as a bare local-dev default, which says nothing about what the sandbox deployment puts in that variable.

The demo UI is a different case. `server.js` has `// ADD USER AND PASSWORD` as a TODO comment next to the `NATS.connect({ servers: natsUrl.url })` call - indicating the original developer intended to add explicit `user/pass` options to the `connect()` call (rather than embed them in the URL) and never did. The demo UI's NATS connection is therefore unauthenticated regardless of what the sandbox NATS server requires - it will fail to connect if the server mandates credentials.

If a username and password are added to the demo UI's NATS connection, this does **not** produce a session token. NATS username/password auth works differently from HTTP session auth: the credential is presented at connection time via the `{ user, pass }` option (or embedded in the URL as `nats://user:pass@host:4222`) and the NATS server accepts or rejects the connection. There is no token issued; the static credential is the entirety of the authentication.

NATS also supports NKey and operator JWT auth (a more secure, token-based approach), but none of the Tazama services currently use this.

NATS also has an optional server-level feature called **subject-level authorisation**, which lets you restrict individual users to only publishing or subscribing on specific subjects. This is not in use in the sandbox. Any connected NATS client can subscribe to any subject. This means the `message.TenantId` payload filter

described in the proposed approach below is not belt-and-suspenders - it is the **only** isolation mechanism between tenants in the real-time layer. Getting that filter right is therefore critical, not optional.

Platform security note: The absence of NATS authentication and subject-level isolation is a platform-wide concern tracked separately in [nats-security-posture.md](#) and [TSC issue #8](#). None of the recommended infrastructure mitigations require changes to the demo UI or any other Tazama service's application code.

Proposed approach:

- The Next.js server (or a dedicated lightweight sidecar service) subscribes to NATS on startup using server-side configuration only.
- Socket.IO connections are authenticated. The browser's NextAuth session cookie (`next-auth.session-token`) is sent automatically with the Socket.IO HTTP upgrade request (same-origin `HttpOnly` cookie). The Socket.IO `io.use()` middleware intercepts the handshake, calls `getServerSession()` (NextAuth v5: `auth()`) to decode and validate the encrypted session cookie, and rejects the connection with an error if no valid session exists. No separate token is needed; the same session mechanism used by the BFF API routes applies here.
- On each authenticated Socket.IO connection, the server fetches the **tenant-specific network map** from admin-service using the user's JWT (`GET /v1/admin/configuration/network_map?filters[active]=true`). admin-service scopes the response to the user's `tenantId` automatically. The full subscription list is then derived from that map: `pub-rule-${rule.id}` for each rule and `typology-${typology.cfg}` for each typology - exactly the logic in `frms-coe-lib/src/helpers/networkMapIdentifiers.ts`.
- Every NATS message in the pipeline carries a `TenantId` field in the `BaseMessage` envelope (`frms-coe-lib/src/interfaces/BaseMessage.ts`). In a shared NATS deployment where multiple tenants run the same rule and typology IDs, subjects are not inherently tenant-prefixed at the rule/typology layer - the demo UI server must filter the NATS message stream by `message.TenantId === tenantId` before emitting to Socket.IO.
- For the three terminal subjects, the actual NATS subject name depends on the `*_DESTINATION` value. When `*_DESTINATION=global` the subject is the bare producer string (e.g., `investigation-service`) and can be subscribed once at server startup, shared across all sessions. When `*_DESTINATION=tenant` the subject is `${PRODUCER}-${tenantId}`, which varies by tenant and is therefore session-specific: it cannot be known until a user connects and their `tenantId` is read from the JWT. The server maintains one NATS subscription per unique active `tenantId` for each tenant-scoped terminal subject, creating the subscription on the first connection from that tenant and releasing it (unsubscribing) when the last session for that `tenantId` disconnects.
- The browser does not need to send any extra credential - the session cookie is attached to the upgrade request automatically by the browser.

Implementation note: On an authenticated Socket.IO connection the server should:

1. Read the user's `tenantId` from the session JWT. Store it in the Socket.IO session state for the lifetime of the connection.
2. Fetch the tenant-specific network map from admin-service using the JWT.
3. Ensure server-wide NATS subscriptions exist for all rule/typology subjects derived from the network map (`pub-rule-${rule.id}` for each rule, `typology-${typology.cfg}` for each typology). These subjects are shared at the message-routing layer across all tenants - the server can maintain a single

subscription per subject name regardless of how many tenant sessions are active, routing messages to the correct sessions via the `TenantId` filter (server-side) and `MsgId` filter (client-side).

4. For each terminal subject: if `*_DESTINATION=global`, ensure a server-startup NATS subscription exists (shared across all sessions). If `*_DESTINATION=tenant`, compute `${PRODUCER}-${tenantId}` using this session's `tenantId`; maintain a reference-counted NATS subscription for each unique active `tenantId` - create it on first connection for that tenant, release it (unsubscribe) when the last session for that `tenantId` disconnects.
5. Filter by `message.TenantId === tenantId` before emitting to Socket.IO. This is the server-side security boundary: it prevents cross-tenant leakage on shared subjects. The server does not need to track which `MsgIds` each session submitted - that filtering happens client-side (see the note on intra-tenant cross-session bleed below).

Intra-tenant cross-session bleed

NATS subjects at the rule/typology layer are not tenant-prefixed - they are derived from the network map and are structurally shared. Even at the terminal layer, when `*_DESTINATION=tenant`, the subject is `${PRODUCER}-${tenantId}`, which is the same for every user within that tenant. There is no NATS-level mechanism to route messages to a specific session or user within a tenant without changes to the producing services.

This means that without mitigation, two concurrent sessions from the same tenant will each receive the other's transaction results via Socket.IO. The server-side `TenantId` filter is a security boundary (cross-tenant isolation), not a per-session one.

The mitigation is client-side `MsgId` filtering. The Demo UI submits a consecutive `pac.008/pac.002` pair to TMS, but only the `pac.002` is transmitted beyond the event-director into the rule/typology pipeline. All rule results, typology scores, and terminal output messages are therefore associated with the `pac.002`'s `MsgId`. The `/api/transaction` route handler must extract this value from the TMS response and return it to the browser, which stores it in React state.

The exact field path differs by pipeline stage:

- **Rule/typology subjects** (`pub-rule-*`, `typology-*`): the pipeline message carries the original transaction in a `transaction` field; the ID is at `message.transaction.FIToFIPmtSts.GrpHdr.MsgId`.
- **Terminal output subjects** (`investigation-service`, etc.): the message is an `Evaluation` object (`frms-coe-lib/src/interfaces/processor-files/TADPReport.ts`) with a top-level `transactionID: string` field. event-adjudicator derives this from `transaction.FIToFIPmtSts.GrpHdr.MsgId` for `pac.002` messages - it is the same value.

Before applying any incoming Socket.IO message to the pipeline display, the browser validates the appropriate field against the stored `MsgId`. If it does not match, the message is discarded without updating the UI. This is the correct layer for this check: it is a UX concern (show the user their own transaction results) rather than a security concern, and the browser is already the right place to own UI state.

3.4 Configuration Architecture

Setting	Current location	Proposed location
TMS API URL	<code>localStorage</code>	Server-side env var (single URL, shared across tenants)
Keycloak OIDC config (<code>KEYCLOAK_ISSUER</code> , <code>KEYCLOAK_CLIENT_ID</code> , <code>KEYCLOAK_CLIENT_SECRET</code>)	Not present (no auth today)	Server-side env vars (NextAuth uses these to perform the OIDC login flow; the client secret is optional if using a public client)
NextAuth session secret (<code>NEXTAUTH_SECRET</code>)	Not present (no auth today)	Server-side env var - required by Auth.js v5 to encrypt the session cookie; Next.js throws at startup if absent. Generate with <code>openssl rand -base64 32</code> . Never expose via <code>NEXT_PUBLIC_</code> .
NATS server URL (+ credentials)	URL sent from browser via <code>socket.on("uiconfig")</code> (SSRF vector); credentials never implemented (<code>// ADD USER AND PASSWORD TODO</code> in <code>server.js</code>)	Single <code>NATS_SERVER_URL</code> env var, server-side only. Credentials embedded in URL if required (<code>nats://user:pass@host:4222</code>), following the same pattern as all other Tazama services. See the NATS authentication discussion in Section 3.3 for current sandbox credential posture.
Alert subject (<code>ALERT_PRODUCER</code> , <code>ALERT_DESTINATION</code>)	Hard-coded <code>investigation-service</code>	Server-side env vars (default: <code>investigation-service</code> / <code>global</code>)
TP interdiction subject (<code>TP_INTERDICTION_PRODUCER</code> , <code>TP_INTERDICTION_DESTINATION</code>)	Hard-coded <code>interdiction-service-tp</code>	Server-side env vars (default: <code>interdiction-service-tp</code> / <code>global</code>)
EF interdiction subject (<code>EF_INTERDICTION_PRODUCER</code> , <code>EF_INTERDICTION_DESTINATION</code>)	Not subscribed	Server-side env vars (default: <code>interdiction-service-ef</code> / <code>global</code>)
DB credentials	<code>NEXT_PUBLIC_PG_*</code> env vars	Eliminated - direct DB access replaced by admin-service API calls
Admin service URL	<code>localStorage</code>	Server-side env var (single URL, shared across tenants)
Condition types/event types	<code>localStorage</code>	admin-service API via BFF (<code>/api/conditions</code>) - fetched server-side using the user's JWT; no env var or database entry needed

The Settings page (`/settings`) is removed entirely. There is nothing left for a user to configure - every setting that previously lived there is now either a server-side env var or has been eliminated. User identity (display

name, tenant ID) and a logout button will be surfaced in the header of the main demo page, alongside the existing controls (e.g. the "Clear All" button). This keeps the authenticated user's context visible without introducing a dedicated page that has no purpose.

4. Tazama 4.0.0 Update

4.1 What Needs Investigation

Before implementation, the following must be reviewed against the `dev` branches of the relevant tazama-lf and frmscoe repositories:

- **Message schemas:** Only `pacs.008.001.10` and `pacs.002.001.12` are relevant - the demo UI does not submit `pain.001` or `pain.013` messages (confirmed: no pain message action types exist in `entity.actions.tsx`). Field names and structures have been verified against `frms-coe-lib 8.0.0-rc.5` type exports - see Q4 below.
- **event-adjudicator result shape: Verified compatible with frms-coe-lib 8.0.0-rc.5.** All field paths used in `tadProcUtils.ts` are unchanged: `report.status` (`Alert.status`), `report.tadpResult.typologyResult` (`TADPResult.typologyResult: TypologyResult[]`), per-`typology .cfg, .result, .ruleResults, .workflow.alertThreshold, .workflow.interdictionThreshold`, and per-rule `.id, .subRuleRef`. No field renaming occurred between 3.0.0 and 8.0.0-rc.5. The only required work is replacing the inline type annotations in `entity.interface.tsx` with imports from `frms-coe-lib`.
- **Admin Service API:** Verified on `dev` (May 2026) - the admin-service exposes all data the demo UI requires with no gaps:
 - `GET /v1/admin/configuration/network_map` - network map structure (`messages[].typologies[].rules[]`) for pipeline rendering; filter by `filters[active]=true` to get the active map (no dedicated active-map endpoint exists despite appearing in the privilege table)
 - `GET /v1/admin/configuration/rule/:id/:cfg` - rule config including `config.bands[].{ lowerLimit, upperLimit, subRuleRef }` for result colour-coding
 - `GET /v1/admin/configuration/typology/:id/:cfg` - typology config including `workflow.{ alertThreshold, interdictionThreshold }` for alert/interdiction state rendering
 - `GET /v1/admin/event-flow-control/entity|account` - conditions (already used)
 - All routes require a valid privilege token. The BFF forwards the authenticated user's JWT (`Authorization: Bearer <jwt>`) to admin-service; admin-service validates it via `extractTenant` and scopes all responses to the user's `tenantId` automatically. This is sufficient because demo UI users belong to a Keycloak group (e.g. `Tazama-Demo`) whose role sub-groups grant the privileges required by both TMS and admin-service (see Section 3.2). No separate service account credential is needed.
 - All paths above are verified from `admin-service v4.0.0-rc.2` (`src/router.ts` + `src/utils/crud-schema.ts`, `dev` branch). No path changes from the 3.0.0 contract.
- **TMS API: Verified - no path changes in 4.0.0.** TMS v4.0.0-rc.1 (`src/router.ts`) exposes the same endpoints the demo UI already calls: `POST /v1/evaluate/iso20022/pacs.008.001.10` and `POST /v1/evaluate/iso20022/pacs.002.001.12`. No investigation needed. The auth model change (removing the direct client-side call in `Debtor.tsx` and routing through the BFF with JWT forwarding) is already documented in Sections 3 and 5.

- **frms-coe-lib version: Resolved - target version is 8.0.0-rc.5.** This is the current `dev` branch HEAD and the version used by rule-executer (the highest across all pipeline services: rule-executer 8.0.0-rc.5, typology-processor 8.0.0-rc.2, event-director 8.0.0-rc.2, event-adjudicator 8.0.0-rc.3). The demo UI currently depends on 6.0.0.

4.2 Known Interface Points

Component	3.0.0 usage	4.0.0 action required
frms-coe-lib	6.0.0 (protobuf decode)	Update to 8.0.0-rc.5; adopt type exports
PACS008 / PACS002	Inline interfaces in <code>entity.interface.tsx</code>	Replace with types from <code>frms-coe-lib</code>
event-adjudicator result	<code>report.tadpResult.typologyResult</code> path (current)	Field paths verified compatible with frms-coe-lib 8.0.0-rc.5 - no renaming needed; replace inline type annotations in <code>entity.interface.tsx</code> with <code>frms-coe-lib</code> imports
TMS endpoint	POST <code>/v1/evaluate/iso20022/pacs.008.001.10</code> and POST <code>/v1/evaluate/iso20022/pacs.002.001.12</code> (direct client call in <code>Debtor.tsx</code>)	Endpoint paths unchanged in TMS v4.0.0-rc.1; move call to BFF and forward Authorization: Bearer <jwt>
Admin Service	REST calls for conditions and network map	Paths verified from admin-service v4.0.0-rc.2 source - all routes in Section 4.1 are confirmed correct; implement as BFF proxies forwarding the authenticated user's JWT (see Section 4.1 for Keycloak group privilege requirements)
NATS topics	<code>investigation-service</code> , <code>interdiction-service-tp</code>	Subject names are env-var-driven - verified in Section 3.3. Default values from <code>docker-compose.hub.relay.yaml</code> are unchanged: <code>ALERT_PRODUCER=investigation-service</code> , <code>INTERDICTION_PRODUCER=interdiction-service-tp/interdiction-service-ef</code> . Replace hard-coded strings with <code>ALERT_PRODUCER /</code> , <code>TP_INTERDICTION_PRODUCER /</code> , <code>EF_INTERDICTION_PRODUCER</code> env vars.
EFRuP rule	Hard-coded <code>EFRuP@1.0.0</code> check in <code>tadProcUtils.ts</code>	When the network map is fetched, scan the rules array for an entry with <code>cfg === 'none'</code> - that is the EFRuP rule. Pass its <code>id</code> into <code>handleAdjudicatorResults()</code> as a

Component	3.0.0 usage	4.0.0 action required
		parameter instead of a hard-coded literal. If no such entry exists (EFRuP not deployed in this network map), treat <code>efrupId</code> as <code>undefined</code> and skip all EFRuP-specific processing - the UI must not assume EFRuP is always present.

4.3 frms-coe-lib Integration

Beyond protobuf decoding, the core library should be leveraged for:

- ISO 20022 message type definitions (`Pacs008`, `Pacs002` from `src/interfaces/Pacs.008.001.10.ts` / `Pacs.002.001.12.ts`) - replace inline interfaces in `entity.interface.tsx`
- Transaction type guards (`isPacs008Transaction`, `isPacs002Transaction` from `src/helpers/transactionTypeGuards.ts`) - use to validate incoming NATS pipeline messages at runtime
- Shared Tazama domain types (`Evaluation`, `Alert`, `TADPResult`, `TypologyResult`, `RuleResult` from `src/interfaces/processor-files/`) - replace inline type annotations in `entity.interface.tsx` and `processor.interface.tsx`
- Network map subject derivation (`getRoutesFromNetworkMap` from `src/helpers/networkMapIdentifiers.ts`) - already referenced in Section 3.3; use directly rather than reimplementing

Note: `src/builders/utils.ts` is Postgres-only (irrelevant to the demo UI). `src/tests/data/pacs008.ts` and `pacs002.ts` are internal test fixtures - not part of the public API and not to be imported.

5. Front-end / Back-end Separation

5.1 Proposed File Structure

```
tazama-demo/
  app/
    (auth)/
      login/                # Login page (URL: /login - consistent with other
Tazama UIs)
    (demo)/
      page.tsx              # Main demo dashboard (authenticated); header includes
user display name, tenant ID, and logout button
      layout.tsx            # Authenticated layout with session check
    api/
      auth/[...nextauth]/   # NextAuth route
      health/               # Health check
      version/              # Version
      network-map/
        route.ts            # GET - proxies to admin-service
/v1/admin/configuration/network_map?filters[active]=true
      rules/
```

```

    route.ts          # GET - proxies to admin-service
/v1/admin/configuration/rule
    typologies/
    route.ts          # GET - proxies to admin-service
/v1/admin/configuration/typology
    transaction/
    route.ts          # POST - proxies to TMS API (forwards session JWT as
Authorization header)
    conditions/
    route.ts          # GET/POST - proxies to admin-service /v1/admin/event-
flow-control
    components/       # Pure UI components (no API calls, no context side-
effects)
    lib/
    auth.ts           # NextAuth configuration
    nats.ts           # Server-side NATS connection manager
    nats-helpers.ts   # Pure filter and subject computation functions
(TenantId filter, MsgId filter, terminal subject name derivation) - extracted for
unit testing (Phase 2 item 10)
    tazama-client.ts  # Typed HTTP client for TMS and Admin Service
(including config endpoints)
    # Note: no tenant.ts URL resolver - TMS and admin-service are universal
endpoints;
    store/            # Client-side state (UI state only, no credentials)
    middleware.ts     # Auth guard - redirect to sign-in if no session
    server.js         # Reduced scope: Socket.IO only, NATS from env vars

```

Note: `lib/db.ts` and the `pg` npm package are **not present** in the updated architecture. The demo UI holds no database credentials.

5.2 BFF Pattern for Transaction Submission

In the current application, the user fills in transaction details (amount, purpose, description, geolocation) via the `TransactionModal` form. On save, `entity.provider.tsx` merges those values into a fully-formed `PACS008` and immediately derives a matching `PACS002` from it via `buildPacs002()`. Both payloads are held in the React context. When the user triggers submission from `components/Device/Debtor.tsx`, the browser makes two direct sequential HTTP calls to TMS - `postPacs008()` first, then `postPacs002()` on success:

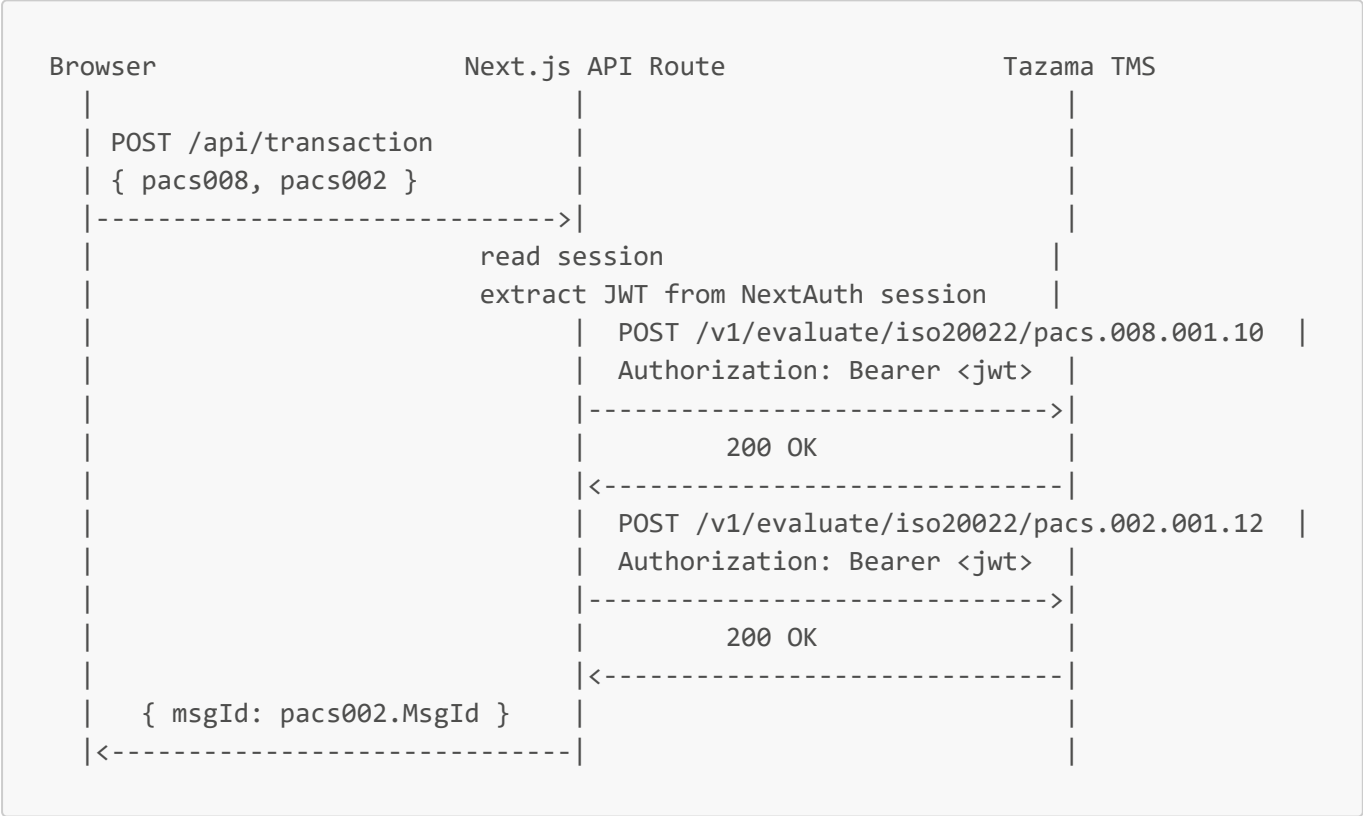
Browser	Tazama TMS
POST /v1/evaluate/iso20022/pacs.008.001.10	
{ pacs008 }	
----->	
200 OK	
<-----	
(800 ms delay)	
POST /v1/evaluate/iso20022/pacs.002.001.12	
{ pacs002 }	
----->	



The pacs.002 `MsgId` is written to `localStorage` as `current_msg_id` immediately when the pacs.002 is built; this value is used by the NATS subscription in `processor.provider.tsx` to filter incoming pipeline result messages.

The problem is that the TMS URL is stored in `localStorage` (set from the Settings page, falling back to `NEXT_PUBLIC_TMS_SERVER_URL`), making it visible to anyone with browser developer tools open. Both payloads and the TMS address are therefore fully exposed client-side.

With the BFF pattern, the browser never knows the TMS URL. It posts both constructed payloads to a local Next.js API route (`/api/transaction`). That route runs on the server, reads the TMS URL from a server-side environment variable, extracts the user's JWT from the NextAuth session, and forwards both requests to TMS in sequence with `Authorization: Bearer <jwt>`. There is no separate TMS API key - TMS authenticates via `extractTenant(AUTHENTICATED, authHeader)` using the same JWT, exactly as every other Tazama forward-facing service does. The browser receives only the pacs.002 `MsgId` needed for NATS filtering.



The browser stores the returned `msgId` in state (replacing the current `localStorage`-based `current_msg_id`) and uses it to filter incoming NATS pipeline messages via the existing subscription logic in `processor.provider.tsx`.

6. Security Hardening

6.1 Next.js Security Vulnerabilities - Upgrade Required

CVE-2025-29927 - Middleware Bypass (March 2025, CVSS 9.1)

An attacker could send a request with a crafted `x-middleware-subrequest` header that tricked Next.js into skipping middleware execution entirely. Any authentication or authorisation logic in `middleware.ts` could be bypassed with a single HTTP header. This was fixed in Next.js [14.2.25](#).

The current `yarn.lock` resolves `next` to [14.2.31](#), so this specific CVE was already patched. However, it established a pattern of critical middleware bypass vulnerabilities that has continued - see the May 2026 release below.

Next.js May 2026 Security Release (May 7, 2026) - Action Required

Vercel published a coordinated release addressing 13 advisories. **Next.js 14.x (all versions) is listed as affected with no 14.x patch available.** The current install ([14.2.31](#)) is vulnerable. The only resolution is to upgrade to **15.5.18** (or 16.2.6).

The advisories most relevant to this codebase:

Advisory	Severity	Why it matters here
Auth bypass via App Router segment-prefetch URL	High	Middleware is the proposed auth enforcement point - bypassing it grants unauthenticated access to everything
App Router segment-prefetch bypass (incomplete fix follow-up)	High	Same - a second bypass vector in the same mechanism
Bypass via dynamic route parameter injection	High	Same family; affects any route using dynamic segments
SSRF in applications using WebSocket upgrades	High	The demo app uses Socket.IO, which negotiates via HTTP upgrade requests. A malicious client could trigger server-side requests to internal hosts.
DoS in React Server Components (CVE-2026-23870)	High	Affects any route using React Server Components, which is the default in the App Router
XSS in App Router applications using CSP nonces	Moderate	Relevant if Content Security Policy headers are added (recommended as part of security hardening)

Vercel explicitly states: "Vercel has not deployed new WAF rules for this release; these advisories cannot be reliably blocked at the WAF layer." Patching is the only complete mitigation.

Upgrade target: Next.js 15.5.18

The upgrade from `14.x` to `15.5.18` must happen in Phase 1. Key breaking changes to address during the upgrade:

Change	Impact on this codebase
<code>params</code> and <code>searchParams</code> in page/layout components are now async (must be <code>awaited</code>)	Affects any page component that reads URL parameters

Change	Impact on this codebase
<code>fetch</code> responses are no longer cached by default	API Route Handlers that call external services will no longer inadvertently cache responses
<code>cookies()</code> and <code>headers()</code> APIs are now async	Affects any server component or route handler that reads cookies (including session checks)
Turbopack is stable for <code>next dev</code>	Dev server should be faster; no code changes needed
React 19 is the default peer dependency	Requires verifying that all third-party UI libraries (Radix UI, headlessui, etc.) are compatible

6.2 OWASP Top 10 Fixes

The following changes directly address OWASP Top 10 vulnerabilities:

Issue	OWASP Category	Fix
DB credentials in <code>NEXT_PUBLIC_*</code> env vars	A02 - Cryptographic Failures	Move to server-only env vars (no <code>NEXT_PUBLIC_</code> prefix)
No authentication	A01 - Broken Access Control	Add NextAuth + Keycloak; guard all routes via middleware
NATS URL controlled by browser	A03 - Injection	NATS config server-side only; remove <code>socket.on("uiconfig")</code>
Credentials in <code>localStorage</code>	A02 - Cryptographic Failures	Move all credentials to server-side config
No CSRF protection on API routes	A01 - Broken Access Control	Use NextAuth session token validation on all mutation routes
Direct database access from API routes	A04 - Insecure Design	Eliminate entirely - replace with admin-service API calls via <code>lib/tazama-client.ts</code> ; the <code>pg</code> package and all DB credentials are removed from the application
<code>crypto</code> npm package (unnecessary)	A06 - Vulnerable Components	Remove; use Node.js built-in <code>crypto</code> if needed
Next.js 14.x - all versions affected by May 2026 security release (middleware bypass, SSRF, DoS, XSS - no 14.x patch)	A01, A06 - Broken Access Control / Vulnerable Components	Upgrade to Next.js 15.5.18 immediately (Phase 1)

7. CI/CD and Workflow Improvements

7.1 Workflow Fixes Required

The canonical `tazama-lf/workflows` repo uses a caller-stub + reusable CI workflow pattern. Several files in `tazama-demo` require updates - three workflows contain old inline logic that must be replaced with canonical caller stubs, two workflows must be copied verbatim (no `workflow_call` trigger), one workflow cannot migrate until the npm migration is complete, and `CODEOWNERS` requires a team-reference update:

- `conventional-commits.yml`: Replace entirely with the canonical caller stub from `tazama-lf/workflows` - it delegates to `conventional-commits-ci.yml@dev` via `uses:`. The current file has full inline logic and is not pinned.
- `gpg-verify.yml`: Replace entirely with the canonical caller stub from `tazama-lf/workflows` - it delegates to `gpg-verify-ci.yml@dev` via `uses:`. The current file has full inline logic and is not pinned.
- `njsscan.yml`: Replace with the canonical caller stub from `tazama-lf/workflows` - it delegates to `njsscan-ci.yml@dev` via `uses:`.
- `check.yml`: Cannot use the canonical `node.js.yml` caller stub without also migrating from `yarn` to `npm` (the canonical `node-ci.yml` reusable uses `npm ci`, `npm run lint:eslint`, `npm run lint:prettier`, and `npm test`). Migration to the canonical stub happens in Phase 0b, after the npm migration in Phase 0a.
- `dockerhub-image-build.yml`: The canonical version cannot be used as a caller stub (it has no `workflow_call` trigger - it is a concrete workflow, not a reusable one). The sync workflow (`sync-workflows.yml`) distributes it automatically after Phase 0b merges: it derives the image name from the repository name (`tazamaorg/${{ env.REPO_NAME }}` via `${{GITHUB_REPOSITORY#GITHUB_REPOSITORY_OWNER/}}`), producing `tazamaorg/tazama-demo`, and reads the release tag from `package.json` version (`type=raw,value=${{ steps.pkg_version.outputs.VERSION }}`). This renames the Docker Hub image from `tazamaorg/demo-ui` to `tazamaorg/tazama-demo`, which is the correct outcome - it aligns repo, image, and container names across all components.
- `dockerhub-image-build-rc.yml`: Same constraint - no `workflow_call` trigger. Distributed automatically by the sync workflow after Phase 0b merges; it already uses `type=raw,value=rc` and derives the image name the same way, producing `tazamaorg/tazama-demo:rc`.
- `CODEOWNERS`: Update from individual GitHub usernames to team references (`@tazama-lf/core-codeowners` `@tazama-lf/community-codeowners`).

8. Implementation Phases

Phase 0 - Prerequisites

Phase 0 consists of two independent PRs that must be merged before the main implementation begins.

Phase 0a - npm migration (tazama-demo)

Standalone PR in `tazama-demo`:

1. Delete `yarn.lock`.
2. Run `npm install` to generate `package-lock.json`.
3. Verify `npm test` passes.
4. Update any scripts that reference `yarn` directly.

This is a prerequisite for Phase 0b: the canonical `node.js.yml` caller stub uses `npm ci` and will fail if the repo still uses `yarn`.

Phase 0b - Add tazama-demo to workflows sync (tazama-lf/workflows)

PR in [tazama-lf/workflows](#) (tracked in [issue #137](#)):

1. Move `tazama-demo` from the excluded "org/non-code repos" list to the active `tazama-lf` array in `release-repo-list.json`.
2. Merge only after Phase 0a is confirmed merged.

After this PR is merged, `sync-workflows.yml` automatically distributes all canonical caller stubs to `tazama-demo`: `conventional-commits.yml`, `gpg-verify.yml`, `njsscan.yml`, `node.js.yml` (replaces `check.yml`), `dockerhub-image-build.yml`, `dockerhub-image-build-rc.yml`, and `CODEOWNERS`. The `node.js.yml` sync also renames the Docker Hub image from `tazamaorg/demo-ui` to `tazamaorg/tazama-demo`.

Phase 1 - Security and Architecture Foundations (highest priority)

1. **[Critical - security]** Upgrade Next.js from `14.2.31` to `15.5.18` to address the May 2026 security release (13 advisories including High-severity middleware bypass, SSRF via WebSocket upgrades, and React Server Components DoS - no 14.x patch exists). Before running the upgrade, remove `next-compose-plugins` from `next.config.js` and `package.json` - this package has not been maintained since the Next.js 10/11 era and is incompatible with Next.js 15; its presence will block the upgrade. After removing it, inline any config composition it was doing directly in `next.config.js`. Then upgrade Next.js, update all `async params/searchParams` usages and `cookies()/headers()` API calls to the async Next.js 15 API, and verify compatibility of Radix UI, headlessui, and other UI library peer dependencies with React 19.
2. Remove all `NEXT_PUBLIC_PG_*` variables and the `pg` npm package entirely; replace the three DB-backed API routes (`/api/network-map`, `/api/rules`, `/api/typologies`) with admin-service proxy calls using `fetch` with the `ADMIN_SERVICE_URL` env var - no JWT forwarding or session validation yet (NextAuth is not installed until Phase 2). The routes make unauthenticated requests to admin-service in Phase 1 and function correctly when `AUTHENTICATED=false`. JWT forwarding and session validation are wired in Phase 2 item 9.
3. Remove the `socket.on("uiconfig")` handler in `server.js`; move NATS URL/credentials to server-side env vars.
4. Remove the Settings page (`/settings`) and all `localStorage` credential storage.
5. Replace boilerplate `middleware.ts` content (remove the Blazity redirect and `// TODO: Feel free to remove this block` comment). Full session-based route protection is added in Phase 2 once NextAuth is in place.

Phase 2 - Authentication

1. Add `@tazama-lf/auth-lib` dependency (same library used by batch-ppa, TMS, and other forward-facing Tazama components).
2. Add `AUTHENTICATED` boolean to the application config (consistent with the pattern in `batch-ppa/src/config.ts`).
3. Add NextAuth.js (Auth.js v5) with Keycloak OIDC provider for the browser-facing login flow (`AUTHENTICATED=true` only). Add `NEXTAUTH_SECRET` as a server-side env var (generate with `openssl rand -base64 32`); Auth.js v5 requires it to encrypt the session cookie and will throw at startup if it is absent (see Section 3.4).
4. Create `lib/auth.ts` with Keycloak provider configuration.

5. Create `/app/(auth)/login/page.tsx` login page modelled on the rule-studio / connection-studio login page design: AppBar header with `tazamaLogo.svg`, two-column layout with the login card (email + password fields, LOGIN button) on the left and `treeImage.png` on the right. The demo UI uses Tailwind rather than MUI, so the layout is a Tailwind equivalent - not a direct port. Copy `tazamaLogo.svg` and `treeImage.png` from `tazama-lf/rule-studio` (both apps share identical assets); add them to `public/`.
6. Update `middleware.ts`: when `AUTHENTICATED=true`, redirect unauthenticated users to `/login` (consistent with other Tazama UIs); when `AUTHENTICATED=false`, pass through without redirect.
7. Create `lib/tazama-client.ts`: typed HTTP client for TMS and admin-service (all configuration endpoints). Centralise all outbound HTTP calls here with shared error handling. This is the foundation used by items 8 and 9 below and Phase 3 item 7.
8. Create the `/api/transaction` BFF route: accepts `{ pacs008, pacs002 }`, validates session via `auth()` (Auth.js v5) when `AUTHENTICATED=true`, extracts JWT from the NextAuth session, forwards both messages to TMS in sequence via `lib/tazama-client.ts` with `Authorization: Bearer <jwt>`, and returns `{ msgId: pacs002.MsgId }` to the browser. No inter-message delay is needed: `pacs.008` is a no-op at event-director and creates no pipeline state that `pacs.002` depends on; the BFF sequential `await` already guarantees `pacs.008` is persisted in TMS (including its Redis data-cache entry, which the `pacs.002` TMS handler reads) before `pacs.002` is submitted.
9. Update all existing BFF API Route Handlers (`/api/network-map`, `/api/rules`, `/api/typologies`, `/api/conditions`): when `AUTHENTICATED=true`, validate session via `auth()` (Auth.js v5 - replaces `getServerSession()` from v4) and return 401 if absent; forward the JWT as `Authorization: Bearer <jwt>` on all calls to admin-service via `lib/tazama-client.ts`. The downstream services handle `tenantId` extraction themselves.
10. Add user display name, tenant ID, and logout button to the main demo page header (logout calls NextAuth `signOut()`).
11. Write unit tests first (RED) for the three pure filter/computation functions that will be implemented in item 12, extracting them into `lib/nats-helpers.ts` before wiring into the Socket.IO integration: (a) `TenantId` filter (`message.TenantId === tenantId`) - covering match, mismatch, and missing-field cases; (b) client-side `MsgId` filter - covering match, mismatch, and missing-field cases for both rule/typology subjects (field path `message.transaction.FIToFIPmtSts.GrpHdr.MsgId`) and terminal output subjects (field path `message.transactionID`); (c) terminal subject name computation (`DESTINATION === 'tenant' → ${PRODUCER}-${tenantId}`, otherwise bare `PRODUCER`) - covering both branches for all three subject types. Green: implement the functions to pass all tests.
12. Implement session-aware NATS subscription for Socket.IO: on connection, when `AUTHENTICATED=true`, validate the session via `auth()` and reject the connection if no valid session exists; when `AUTHENTICATED=false`, skip validation and use `tenantId = 'DEFAULT'` (consistent with all other Tazama components). After resolving the `tenantId`, fetch the tenant-specific network map from admin-service using the user's JWT (or without auth when `AUTHENTICATED=false`); derive NATS subscription subjects using `getRoutesFromNetworkMap()` (`pub-rule-${rule.id}`, `typology-${typology.cfg}`); implement reference-counted per-tenant subscriptions for terminal subjects; apply the `TenantId` and `MsgId` filter functions from item 11 before emitting to Socket.IO (see Section 3.3).

Phase 3 - Tazama 4.0.0 API Update

1. Update `frms-coe-lib` dependency to `8.0.0-rc.5`.
2. Replace inline ISO 20022 type interfaces in `entity.interface.tsx` with `Pacs008` and `Pacs002` imports from `frms-coe-lib`.

3. Replace inline domain type annotations in `entity.interface.tsx` and `processor.interface.tsx` with `Evaluation`, `Alert`, `TADPResult`, `TypologyResult`, and `RuleResult` imports from `frms-coe-lib`.
4. Add runtime NATS message validation using `isPacs008Transaction` and `isPacs002Transaction` type guards from `frms-coe-lib/src/helpers/transactionTypeGuards.ts`; discard malformed messages before they reach the pipeline display.
5. Write unit tests first (RED) for EFRuP detection in `tadProcUtils.ts`, covering three cases: (a) network map containing one entry with `cfg === 'none'` - must return that rule's `id`; (b) network map with no `cfg === 'none'` entry - must return `undefined` and all EFRuP-specific branches in `handleTadProcResults` must be skipped; (c) network map with multiple rules where one has `cfg === 'none'` - must return the correct entry. Green: replace the hard-coded `'EFRuP@1.0.0'` string with a parameter derived from the fetched network map (scan rules for `cfg === 'none'`; pass as `efrupId`; skip EFRuP-specific processing when absent).
6. Before renaming, write characterization tests (RED) capturing the current behavior of `handleTadProcResults` and `handleTadProcLive` in `tadProcUtils.ts` - these lock in the existing logic and act as a regression harness through the rename. Then rename all TADProc-specific identifiers throughout the codebase to reflect the event-adjudicator service rename: rename `utils/tadProcUtils.ts` → `utils/adjudicatorUtils.ts` and update its import in `processor.provider.tsx`; rename `handleTadProcResults` → `handleAdjudicatorResults` and `handleTadProcLive` → `handleAdjudicatorLive`; rename state properties `tadProcResults` → `adjudicatorResults`, `tadpLights` → `adjudicatorLights`, `tadprocLoading` → `adjudicatorLoading`; rename Redux action type strings `UPDATE_TADPROC_*` / `SET_TADPROC_RESULTS` / `RESET_TADPROC_RESULTS` → `UPDATE_ADJUDICATOR_*` / `SET_ADJUDICATOR_RESULTS` / `RESET_ADJUDICATOR_RESULTS`; rename the Socket.IO event name `"tadProc"` → `"eventAdjudicator"` in `server.js`, `processor.provider.tsx`, and `page.tsx`; update the UI display label `"Tadproc"` → `"Event Adjudicator"` in `page.tsx`. Verify all characterization tests remain green after the rename. Note: the `TADPROC` / `TADPROC_RESULT` interfaces in `processor.interface.tsx` are already being replaced by `frms-coe-lib` exports in items 2-3 above and should be removed rather than renamed. The `frms-coe-lib` field name `tadpResult` on the `Alert` interface is a library type name and is not affected by this rename.
7. Implement the admin-service BFF proxy routes for conditions, network map, rules, and typologies using `lib/tazama-client.ts`, forwarding the authenticated user's JWT to admin-service. All API paths are verified correct from admin-service v4.0.0-rc.2 (Section 4.1); no path changes are needed.
8. Update `components/Device/Debtor.tsx`: remove the direct `axios.post` calls to TMS; call `/api/transaction` instead and store the returned `msgId` in React state (replacing `localStorage`-based `current_msg_id`).
9. Bump `package.json` version to `4.0.0`.

Phase 4 - Code Quality

1. Remove unused boilerplate dependencies: `crypto` npm package, `storybook`, `eslint-plugin-storybook`, and all related Storybook packages and scripts (`storybook`, `test-storybook`, `build-storybook` from `package.json`). Note: `next-compose-plugins` is removed in Phase 1 item 1 as a prerequisite to the Next.js upgrade.
2. Add proper error boundaries to client components.
3. Add Husky with lint-staged and commitlint. ESLint and Prettier are already configured in the codebase; Husky, lint-staged, and commitlint are absent. Add `husky` and `lint-staged` (pre-commit hook running

ESLint + Prettier on staged files) and `@commitlint/cli` + `@commitlint/config-conventional` (commit-msg hook for conventional commit enforcement, consistent with all other Tazama repos).

Phase 5 - Testing

1. Complete unit test coverage for any logic in `adjudicatorUtils`, `helpers`, and BFF route handlers not already covered by the test-first steps in Phases 2 and 3. Jest, `@testing-library/react`, `ts-jest`, and `jest-environment-jsdom` are already installed; the `test` script runs `jest --passWithNoTests`. No additional test infrastructure is needed.
2. Playwright e2e test suite - tracked in [tazama-lf/tazama-demo#80](#). Implement after Phase 3 is stable.

9. Open Questions

The following items require decisions or input before implementation can begin:

#	Question	Impact
Q1	Resolved - The Keycloak instance is a deployment concern. The demo UI reads <code>KEYCLOAK_ISSUER</code> from server-side env vars and is agnostic to which Keycloak server it points at. Keycloak is a standard component of the Tazama full stack; set <code>KEYCLOAK_ISSUER=http(s)://<keycloak-host>/realms/Tazama</code> for the target deployment.	-
Q2	Resolved - Single <code>Tazama</code> realm, shared across all tenants on the deployment's Keycloak server. Users belong to a group hierarchy (functional group → role sub-group → tenant sub-group → individual user). Keycloak maps group membership to a <code>tenantId</code> JWT claim; the demo UI reads that claim via <code>@tazama-lf/auth-lib</code> 's <code>extractTenant</code> . No dynamic issuer selection is needed; a single <code>KEYCLOAK_ISSUER</code> pointing at the <code>Tazama</code> realm suffices.	-
Q3	Resolved - env vars, as established in Section 3.4. The Settings page is removed; there is no admin-configurable backend URL store. All service URLs are server-side env vars set at deployment time.	-
Q4	Resolved - target version is <code>8.0.0-rc.5</code> (current <code>dev</code> branch HEAD of <code>frms-coe-lib</code> ; matches rule-executer, the highest version across all pipeline services). Demo UI currently uses <code>6.0.0</code> .	Phase 3
Q5	Resolved - no confirmation needed. The terminal subject env vars (<code>ALERT_PRODUCER</code> , <code>ALERT_DESTINATION</code> , <code>TP_INTERDICTION_PRODUCER</code> , <code>TP_INTERDICTION_DESTINATION</code> , <code>EF_INTERDICTION_PRODUCER</code> , <code>EF_INTERDICTION_DESTINATION</code>) are set at deployment time. The code reads whatever values are configured and applies the <code>*_DESTINATION === 'tenant'</code> subject computation formula described in Section 3.3. The implementer sets sensible defaults (<code>investigation-service</code> / <code>interdiction-service-tp</code> / <code>interdiction-service-ef</code> , all <code>global</code>) and the deployer overrides as needed for the sandbox.	-
Q6	Resolved - EFRuP is present in the 4.0.0 network maps at <code>{ id: 'EFRuP@1.0.0', cfg: 'none' }</code> (confirmed from event-flow <code>dev</code> branch). The EFRuP rule ID is derived from the fetched network map rather than hard-coded; the UI gracefully skips EFRuP-specific	Phase 3

#	Question	Impact
	rendering when no <code>cfg === 'none'</code> entry is present, making it deployable against both event-flow and non-event-flow network maps.	
Q7	Resolved - the Settings page is removed entirely. All configuration is server-side env vars. User identity (display name, tenant ID) and logout are surfaced in the main demo page header.	-
Q8	Resolved - migrate to <code>npm</code> . Standalone Phase 0a PR in <code>tazama-demo</code> (delete <code>yarn.lock</code> , run <code>npm install</code> , verify <code>npm test</code> , update scripts) before all other phases.	Phase 0
Q9	Resolved - remove. No <code>.storybook</code> config and no <code>.stories.*</code> files exist; Storybook was pulled in from the Next.js boilerplate template and never used. Remove the <code>storybook</code> , <code>eslint-plugin-storybook</code> , <code>test-storybook</code> , and related packages and scripts.	Phase 4
Q10	Resolved - the demo UI only submits <code>pacs.008.001.10</code> and <code>pacs.002.001.12</code> . No <code>pain.001</code> or <code>pain.013</code> action types exist in <code>entity.actions.tsx</code> . The demo UI does not need to support pain message flows.	Phase 3
Q11	Resolved - verified from <code>frms-coe-lib/networkMapIdentifiers.ts</code> , event-director, rule-executer, typology-processor, and event-adjudicator. NATS subjects for rule and typology results are derived deterministically from the network map (not tenant-prefixed at the message-routing layer). All pipeline messages carry <code>TenantId</code> in the <code>BaseMessage</code> payload. Terminal output subject isolation is optional via <code>ALERT_DESTINATION</code> / <code>INTERDICTION_DESTINATION</code> . The demo UI derives its subscription list from the network map and filters by payload <code>TenantId</code> . See Section 3.3 and updated Q5 for remaining operational question.	-
Q12	Resolved - NATS subject-level authorisation is not in use in the sandbox. Any connected NATS client can subscribe to any subject. The <code>message.TenantId</code> payload filter (Section 3.3, implementation note step 5) is therefore the sole cross-tenant isolation mechanism in the real-time layer and must be implemented correctly.	-

10. Post-Review Follow-up Questions / Gaps

Identified during a full top-to-bottom document review on 2026-05-30. Items marked **Resolved** have been applied to the document; open items need a decision before the relevant phase begins.

#	Gap	Status	Phase
A	Admin-service credential contradiction - Phase 2 item 8 said to forward the user's JWT to admin-service; Section 4.1 said the opposite (service account credential).	Resolved - The service account approach was modelled on TCS/TRS (server-to-server, no user sessions). The demo UI has authenticated user sessions, making the user's own JWT the natural and sufficient credential for both TMS and admin-service. Demo users must belong to a Keycloak functional group (e.g. <code>Tazama-Demo</code>) whose role sub-groups grant the privileges required by both	-

#	Gap	Status	Phase
		APIs. The BFF forwards the user's JWT to all downstream services; no separate service account is needed. Document updated in Sections 3.2, 4.1, 4.2, Phase 1 item 2, and Phase 3 item 7.	
B	Service account credential: no env var name or format specified - "Server-side service account credential" was referenced in multiple places with no env var name, format, or provisioning steps documented.	Resolved - Eliminated by the resolution of gap A. No service account credential is needed; the user JWT is the only credential the BFF forwards.	-
C	next-compose-plugins removal in wrong phase - This package has not been maintained since the Next.js 10/11 era and will not work with Next.js 15. It is currently scheduled for removal in Phase 4 item 2, but its presence will block the Phase 1 upgrade. It must be removed as part of Phase 1 item 1 (or as an explicit prerequisite step within it).	Resolved - Phase 1 item 1 updated to remove next-compose-plugins before the upgrade and inline any composed config directly in next.config.js . Removed from Phase 4 item 2 with a cross-reference note.	Phase 1
D	NEXTAUTH_SECRET env var missing - Auth.js v5 requires NEXTAUTH_SECRET (or AUTH_SECRET) for encrypting the session cookie. Without it the auth layer will not function and Next.js will throw at startup. Not mentioned in Section 3.4, Phase 2, or anywhere else. Must be added to the configuration table and Phase 2.	Resolved - Added to Section 3.4 configuration table and Phase 2 item 3. NEXTAUTH_SECRET is a server-side env var generated via openssl rand -base64 32 ; must never use a NEXT_PUBLIC_ prefix.	Phase 2
E	getSession() vs auth() - Auth.js v5 replaces getSession() with auth() . The NATS section already notes this correctly. Phase 2 items 7 and 8 still use getSession() without the qualifier, which may cause confusion during implementation.	Resolved - Phase 2 items 7 and 8 updated to use auth() with a parenthetical note that it replaces getSession() from Auth.js v4.	Phase 2
F	Section 3.4 config table: "Condition types/event types" row - Proposed location still says "Server-side env var or database". The actual decision (admin-service API via BFF	Resolved - Config table row updated to "admin-service API via BFF (/api/conditions) - fetched server-side using the user's JWT; no env var or database entry needed".	Phase 3

#	Gap	Status	Phase
	<code>/api/conditions</code>) is documented in Section 4.1 and Phase 3 item 7 but was not reflected in the configuration table.		
G	Section 5.1 file structure: <code>lib/nats-helpers.ts</code> missing - Phase 2 item 10 (TDD step) explicitly creates <code>lib/nats-helpers.ts</code> to hold the NATS filter and subject computation functions. It is absent from the proposed file structure tree in Section 5.1.	Resolved - Added <code>lib/nats-helpers.ts</code> to the Section 5.1 file structure tree with a comment describing its contents and its origin in Phase 2 item 10.	Phase 2
H	Phase 2 item 11: <code>AUTHENTICATED=false</code> behaviour not defined - Item 11 says "on connection, validate the session" but does not state what happens when <code>AUTHENTICATED=false</code> (no session exists). Expected behaviour: skip validation and use <code>tenantId = 'DEFAULT'</code> for NATS message filtering, consistent with all other Tazama components.	Resolved - Phase 2 item 11 updated to explicitly branch on <code>AUTHENTICATED</code> : when <code>true</code> , validate via <code>auth()</code> and reject if absent; when <code>false</code> , skip validation and use <code>tenantId = 'DEFAULT'</code> .	Phase 2
I	Section 7.1 intro text: "Three workflows" - The intro says "Three workflows in tazama-demo contain old inline logic" but the section then lists seven items (conventional-commits, gpg-verify, njsscan, check.yml, dockerhub-image-build.yml, dockerhub-image-build-rc.yml, CODEOWNERS). Minor accuracy issue.	Resolved - Intro rewritten to accurately describe the seven items: three inline-logic replacements, two verbatim copies (no <code>workflow_call</code> trigger), one deferred migration (npm prerequisite), and one CODEOWNERS update.	Phase 1
J	Section 3.1 stale "service account" text - The third paragraph of Section 3.1 still says "all configuration data ... is retrieved from the admin-service API using a server-side service account." Gap A resolved that no service account is needed - the user's JWT is forwarded instead. The intro text was never updated.	Resolved - Section 3.1 updated to "forwarding the authenticated user's JWT; admin-service scopes all responses to the user's <code>tenantId</code> automatically". Also removed the stale TCS/TRS comparison sentence (that pattern uses a service account; the demo UI does not).	Phase 1

#	Gap	Status	Phase
K	Section 3.2 auth flow diagram still uses <code>getServerSession()</code> - Gap E updated Phase 2 items 7 and 8, but the inline pseudocode block in the Section 3.2 <code>AUTHENTICATED=true</code> flow still reads <code>validates session exists via getServerSession()</code>. Should be <code>auth()</code> (Auth.js v5) to match the rest of the document.	Resolved - Section 3.2 auth flow step 4 updated to <code>auth()</code> (Auth.js v5).	Phase 2
L	Sign-in page URL mismatch: <code>/auth/signin</code> vs <code>/signin</code> - The file structure (Section 5.1) places the sign-in page at <code>app/(auth)/signin/</code>. In Next.js App Router, route group names in parentheses are excluded from the URL, so this produces <code>/signin</code> not <code>/auth/signin</code>. Section 3.2 step 1 and Phase 2 item 6 both redirect to <code>/auth/signin</code>, which would 404. Either the file path must be <code>app/auth/signin/</code> (no parens, URL becomes <code>/auth/signin</code>) or the redirect target must change to <code>/signin</code>.	Resolved - Use <code>/login</code> (consistent with other Tazama UIs: rule-studio, connection-studio). File path: <code>app/(auth)/login/page.tsx</code> ; middleware and Section 3.2 redirect target updated to <code>/login</code> .	Phase 2
M	<code>lib/tazama-client.ts</code> phasing: used before it is created - Phase 2 item 7 and Phase 3 item 7 both reference <code>lib/tazama-client.ts</code>. Phase 4 item 1 is the step that creates it ("Centralise all Tazama API HTTP calls in <code>lib/tazama-client.ts</code>"). The library is consumed two phases before it exists. Either Phase 4 item 1 needs to move to Phase 2 (before item 7), or Phase 2 and 3 need a note that they stub the HTTP calls inline and Phase 4 item 1 is the consolidation step.	Resolved - Phase 4 item 1 moved to Phase 2 item 7 (before the BFF routes that use it). Phase 4 renumbered to 3 items. Phase 2 items 7-11 renumbered to 8-12.	Phase 2
N	Phase 1 item 2: JWT forwarding before auth exists - Phase 1 item 2 says to replace DB-backed routes with admin-service proxy calls "forwarding the user's JWT (the BFF validates the	Resolved - Phase 1 item 2 updated: DB layer removed and admin-service proxy wired with unauthenticated requests only; JWT forwarding and session validation deferred to Phase 2 item 9.	Phase 1

#	Gap	Status	Phase
	session exists first)". NextAuth is not installed until Phase 2 - there is no JWT and no session in Phase 1. Phase 1 can only remove the database calls and stub the proxy structure; JWT forwarding and session validation cannot be wired until Phase 2 auth is in place.		
O	800ms inter-message delay not captured in Phase 2 item 7 - The current <code>Debtor.tsx</code> inserts an 800ms delay between the pacs.008 and pacs.002 submissions (visible in the Section 5.2 BFF diagram as "(800 ms delay)"). Phase 2 item 7 creates the <code>/api/transaction</code> BFF route but does not mention preserving this delay. If omitted, pacs.002 will arrive at event-director before pacs.008 has cleared the pipeline, breaking the demo's visual flow.	Resolved - The delay premise was incorrect. pacs.008 is a no-op at event-director - it creates no pipeline state that pacs.002 depends on, so there is no race condition to prevent at the NATS/pipeline layer. The only real ordering requirement is at the TMS DB layer (pacs.002's TMS handler reads pacs.008's Redis data-cache entry), which is already guaranteed by the BFF sequential <code>await</code> : TMS awaits all DB writes before returning 200 for pacs.008, and the BFF awaits that 200 before submitting pacs.002. The 800ms delay is removed from Phase 2 item 8. The <code>(800 ms delay)</code> annotation in the Section 5.2 current-behaviour diagram is retained as an accurate description of the existing code.	Phase 2
P	Phase 0b sync vs Section 7.1 "copy verbatim": ambiguous ownership of dockerhub workflows - Section 7.1 says <code>dockerhub-image-build.yml</code> and <code>dockerhub-image-build-rc.yml</code> must be "copied verbatim" (implying manual action). Phase 0b says <code>sync-workflows.yml</code> "automatically distributes ... <code>dockerhub-image-build.yml</code> , <code>dockerhub-image-build-rc.yml</code> " (implying automation). These cannot both be the authoritative description. One must be the single source of truth and the other removed or qualified.	Resolved - Phase 0b automation is authoritative. Section 7.1 updated: "copy verbatim" replaced with "distributed automatically by the sync workflow after Phase 0b merges" for both dockerhub workflow entries.	Phase 0